The codeverbatim and codehighlight Packages
Version 1.0

Alceu Frigeri*

November 2026

Abstract

These packages enables the construction of custom verbatim commands/environments with syntax highlight, splitting the action of grabbing some text in verbatim mode (*codeverbatim*) from typesetting it with or without syntax highlight (*codehighlight*). That way it's possible to fully customize the final desired behaviour, including (pre/post) processing of the grabbed text before typesetting it.

Contents

1 Introduction **2**

2 codeverbatim Package **3**
- 2.1 Characters Category Code and Common Keys . . . 3
- 2.2 New Verbatim Environment . . . 3
- 2.3 Auxiliary Environment Begin Commands . . . 4
- 2.4 Auxiliary Store Commands . . . 4
- 2.5 Auxiliary Display Commands . . . 6
- 2.6 Short Verbatim Commands . . . 6
  - 2.6.1 Caveats . . . 7

3 codehighlight Package **9**
- 3.1 UTF8 and other active characters . . . 9
- 3.2 Setting defaults . . . 9
- 3.3 Display commands . . . 10
- 3.4 Language and syntax highlight definitions . . . 10
- 3.5 Based on syntax and class remapping . . . 11
- 3.6 Internal structures and scan functions . . . 11
- 3.7 Some package settings . . . 12
- 3.8 Base Format . . . 12
- 3.9 Highlight Sets . . . 13
- 3.10 Auxiliary Format Commands . . . 14
- 3.11 Keywords Sets . . . 14
- 3.12 Language Syntax . . . 15
- 3.13 Aliases . . . 18
- 3.14 Pre-defined Languages . . . 18

4 Examples **20**
- 4.1 A minimal construct . . . 20
- 4.2 Customizing the language . . . 20
- 4.3 Further customization . . . 21
- 4.4 Using sequences . . . 21

5 To do, Future plans **22**

*https://github.com/alceu-frigeri/codeverbatim

1 Introduction

The main aim of these packages is to allow the creation of fully customizable verbatim environments. To achieve that, unlike other packages of the kind as *fancyvrb* [6], *scontents* [2] or *listings* [3], which have a fixed set of ways to collect/display a verbatim environment, *codeverbatim* and *codehighlight* split the action of grabbing some text in verbatim mode (*codeverbatim*) from typesetting it with or without syntax highlight (*codehighlight*).

codeverbatim package offers a mechanism to create custom verbatim environments whereas the grabbed text will be stored either in a *expl3* token list or sequence variables with hooks allowing the customization of the entire process.

> ***Note:*** A great deal of the functionality of this is inspired on the *scontents* [2] package.

codehighlight package offers ways to (syntax) highlight the code grabbed by a *codeverbatim* environment. It's somewhat inspired on the *listings* [3] functionality, allowing the definition of a custom syntax (for code highlight). Unlike *listings*, in *codehighlight* four aspects: (1) syntax definition, (2) keyword's sets, (3) highlight formatting and (4) base formatting, are kept separated, allowing a more flexible setup.

> ***Note:*** Although the two packages are though to be used together, they don't rely on each other. It's perfectly possible to use *codeverbatim* together with *listings* package, or use the *codehighlight* with other verbatim package, like *fancyvrb* (using the macros `\SaveVerb` and `\UseVerb`, albeit not straightforward).

Lastly, these packages are fully written in *expl3*, but given that one might want to use then in a LaTeX 2$_\varepsilon$ context (either at package or document level), all public commands have a LaTeX 2$_\varepsilon$ alias, and, for simplicity, they are listed together.

2 codeverbatim Package

The aim of this package is to allow to construction of custom environments and commands to capture verbatim content. Among others, one can customize:

- which characters should be captured with category code other.
- how to handle the text after the environment begin and the first end of line.
- how to process the captured code.

Another point is that this package only offers ways to capture content in verbatim mode, leaving it to the user to define how to postprocess and display it, for instance, using *codehighlight* (see 3) package.

2.1 Characters Category Code and Common Keys

When capturing content in verbatim mode, many characters have their usual category code changed to other (12), but not all. In particular `^^M` (char: 13, End Of Line character) always has its category code changed to other.

`\`	(char: 92)	`$`	(char: 36)	`^^A`	(char: 1)
`{`	(char: 123)	`^`	(char: 94)	`^^I`	(char: 9)
`}`	(char: 125)	`_`	(char: 95)	`^^J`	(char: 10)
`#`	(char: 35)	`&`	(char: 38)	`^^K`	(char: 11)
`%`	(char: 37)	`~`	(char: 126)		(char: 32, space)

Note: This list is named "other characters list" in the following (for simplicity).

Note: As noted above, `^^M` will always have its category code changed to other (12). Moreover, following the `\tl_retokenize:n` convention, lines will be terminated by `^^J` (and not `^^M`), also with category code other. See the chapter "Reassigning category codes" (under "Manipulating tokens in token lists") from *The LaTeX3 Interfaces* [5], for details.

By default no other character has its category code changed. This can be changed with the keys below, in which there are two cases:

(i) List of characters set to other: keys *set other* and *other*,

(ii) List of characters set to letter: key *letter*,

These keys are common to all main code verbatim commands and expect a comma separated list of characters:

`set other` This will reset and fully replace the "other characters list". If used more than once, only the last attribution remains.

The default is the "other characters list" above. Don't forget to escape the characters as needed, for instance, like `\^^K`

`other` (default: empty list) This will add characters to the "other characters list" and can be called multiple times.

`letter` (default: empty list) The characters on the list (cumulative list) will be made letter.

Note: These character lists shall contain a single char per item! and they will raise an error if more than one character (per item) is found.

Attention If all keys are given, first *other* then *letter* will be applied.

2.2 New Verbatim Environment

`\cvNewEnvironment`
`\codeverb_new_environment:nnnnn`

`\cvNewEnvironment` [⟨keys⟩] {⟨env-name⟩} {⟨arg spec⟩} {⟨begin-code⟩} {⟨end-code⟩}
`\codeverb_new_environment:nnnnn` {⟨keys⟩} {⟨env-name⟩} {⟨signature⟩} {⟨begin-code⟩} {⟨end-code⟩}

This will create a new document environment ⟨env-name⟩, following the `\NewDocumentEnvironment` structure. ⟨arg spec⟩ is the desired ⟨env-name⟩ parameter specification list (`\NewDocumentEnvironment` 1st parameter, see [4]). The ⟨begin-code⟩ code will be executed before the verbatim code gets grabbed. The ⟨end-code⟩ will be executed right after, see the keys *inline* and *verb tl* below for details on how to further handle the verbatim content.

Note: If short verbatim commands are on (`\cvSVerbOn` or `\codeverb_sverb_on:`) those active characters will be made other (see 2.6.1) after applying the characters category code defined here (with the common keys from 2.1) and before applying the characters category code defined with `\cvEnvCCSet` or `\codeverb_env_catcode_set:n` (see 2.3 below).

⟨keys⟩ can be any combination of (besides the ones from 2.1):

`drop first` (default: false) If the character following the environment begin until the line end shall be processed or discarded. The default behaviour is similar to other packages of the kind, like *listings* or *fancyvrb*

`drop blanks` (default: false) If set to true, and the initial lines are blank, those lines will be discarded.

`line proc` (default: -nothing-) The given command shall grab 2 parameters. It will receive a single line (verbatim material) as the 2nd parameter and a temporary token list as the 1st parameter. The simplest command would be `\tl_set:Nn` (no pre-processing at all). This can be used to pre-process the verbatim content, one line at a time.

`inline` (default: false) If set, ⟨end-code⟩ will receive the verbatim material as `#1`. This has precedence over the key *verb tl*.

`verb tl` (default: `\g_codeverb_verb_body_tl`) If *inline* is set to false, the verbatim material will be stored in the given token list variable, before ⟨end-code⟩ is executed. The assignment is global.

Off course, the default is ok to be used in an *expl3* code régime, you may want to change that.

2.3 Auxiliary Environment Begin Commands

`\cvEnvCCSet` `\cvEnvCCSet {⟨keys⟩}`
`\codeverb_env_catcode_set:n` `\codeverb_env_catcode_set:n {⟨keys⟩}`

These are meant to be used in the ⟨begin-code⟩ of a custom environment, allowing for further (local) customization of characters category code. ⟨keys⟩ can be any combination of the common keys (see 2.1). For example:

```
\cvNewEnvironment[drop first]{myEnv}{O{}}
  { \cvEnvCCset {#1} }
  { }
...

\begin{myEnv}[letter = { @ , \_ }]
  some text
\end{myEnv}
```

Note that the environment *myEnv* was defined with the default verbatim setup (category codes as in 2.1) and, in this example, with a single optional parameter to allow changing the characters category code by the end user.

Note: It won't replace the environment defaults, but will be applied after the environment setup.

Attention Unlike in all other cases, the default "other characters list" for these commands is an empty list. That way, it will only change the catcode (to other or letter) of the given characters.

2.4 Auxiliary Store Commands

Besides just storing the verbatim content in a token list variable (key *verb tl* above) one has a few other options:

(i) store the verbatim content in dedicated sequence variables (similar to *scontents* package [2]),
(ii) display them using another package (like *listings*[3]) or
(iii) use them as latex code right away. The following commands aim at exactly that.

\cvNewStoreCommands `\cvNewStoreCommands [⟨key⟩] {⟨cs-base⟩}`
\codeverb_new_store_cmds:nn `\codeverb_new_store_cmds:nn {⟨key⟩} {⟨cs-base⟩}`

These will create a set of commands prefixed with ⟨cs-base⟩, using a set of dedicated *expl3* sequence variables to help handle verbatim code in a way similar to how the package *scontents* [2] does. By default, two *expl3* commands and two LaTeX 2_ε aliases will be created. This can be changed with the following key:

`expl3` (default: false) If set, only the *expl3* variants will be created and no LaTeX 2_ε aliases will be defined.

\<cs-base>PutRightnn `\<cs-base>PutRightnn {⟨st-name⟩} {⟨verb-content⟩}`
\<cs-base>Merge `\<cs-base>Merge {⟨var⟩} {⟨base-list⟩} {⟨index-list⟩}`
\<cs-base>_gput_right:nn `\<cs-base>_gput_right:nn {⟨st-name⟩} {⟨verb-content⟩}`
\<cs-base>_merge:Nnn `\<cs-base>_merge:Nnn {⟨var⟩} {⟨base-list⟩} {⟨index-list⟩}`

The `\<cs-base>PutRightnn` (or `\<cs-base>_gput_right:nn`) will create (as needed) an internal sequence variable associated with ⟨st-name⟩ and store ⟨verb-content⟩ in it. Note, three other commands variants will be defined: `\<cs-base>PutRightnV`, `\<cs-base>PutRightVV` and `\<cs-base>PutRightVn`, all expecting 2 parameters, the `V` denoting if the first, second or all parameters are stored in an auxiliary ⟨tl-var⟩ (LaTeX 2_ε command).
The `\<cs-base>Merge` (or `\<cs-base>_merge:Nnn`) will merge items of one, or more, of those sequences.

The syntax choose resembles the one from the command `\mergesc` package *scontents*, and there are two main (proper) cases:

- ⟨index-list⟩ is an empty list.

  In this case ⟨base-list⟩ is expected to be a list of the kind ⟨st-name⟩ [⟨index-ref⟩], where ⟨index-ref⟩ can be a single number (the sequence index) or a range like ⟨index⟩-⟨index⟩. 1 is the index of the first item, while *end* can be used to reference the last item of a sequence, so that, ⟨st-name⟩[2-end] will refer all items starting from the 2nd. If no [⟨index-ref⟩] is given, then the last item of ⟨st-name⟩ will be used.

- ⟨index-list⟩ isn't empty.

  In this case ⟨base-list⟩ is expected to be a list of ⟨st-name⟩s (no indexes given), while ⟨index-list⟩ is expected to be a list of just indexes, then all indexes from ⟨index-list⟩ will be applied to each ⟨st-name⟩ from ⟨base-list⟩.

For example:

```
\cvNewStoreCommands {myStore}
...
... many calls to \myStorePutRight { case-A } { -content- }
...
... many calls to \myStorePutRight { case-B } { -content- }

\myStoreMerge { \tempA } { case-A[1] , case-B[2] , case-A[2-end] , case-B }

\myStoreMerge { \tempB } { case-A , case-B } { 1 , 3-end }
```

`\tempA` will end up with the 1st item from ⟨case-A⟩, then the 2nd item from ⟨case-B⟩, followed by all items from the 2nd to the last one from ⟨case-A⟩, and finally the last one from ⟨case-B⟩.

`\tempB` will end up with the 1st item from ⟨case-A⟩ followed by all items from the 3rd to the last one from ⟨case-A⟩, then the 1st item from ⟨case-B⟩ followed by all items from the 3rd to the last one from ⟨case-B⟩.

\cvRetokenEnvNewCommand `\cvRetokenEnvNewCommand [⟨key⟩] {⟨env-cmd⟩} {⟨env-name⟩}`
\codeverb_retoken_env_newcs:nn `\codeverb_retoken_env_newcs:nnn {⟨key⟩} {⟨env-cmd⟩} {⟨env-name⟩}`

These will create a set of commands (see below) tailored at retokenizing some (verbatim) content inside the given environment ⟨env-name⟩. By default, two *expl3* commands and two LaTeX 2_ε aliases will br created. This can be changed with the following key:

`expl3` (default: false) If set, only the *expl3* variants will be created and no LaTeX 2_ε aliases will be defined.

`\<env-cmd>`	`\<env-cmd>` {⟨verb-content⟩}
`\<env-cmd>V`	`\<env-cmd>V` {⟨tl-var⟩}
`\<env-cmd>:n`	`\<env-cmd>:n` {⟨verb-content⟩}
`\<env-cmd>:V`	`\<env-cmd>:V` {⟨tl-var⟩}

`\<env-cmd>` (viz-à-viz `\<env-cmd>:n`) will re-scan (retokenize) ⟨verb-content⟩ inside an ⟨env-name⟩ environment. Similarly, `\<env-cmd>V` (viz-à-viz `\<env-cmd>:V`) will re-scan the contents (single expansion) of the ⟨tl-var⟩ inside the ⟨env-name⟩ environment. So that, for example:

```
\lstnewenvironment{lstenv}
  {
    % listings setup
  }
  {}

\cvRetokenEnvNewCommand
  {VerbListing} {lstenv}
```

once defined, `\VerbListingV \g_codeverb_verb_body_tl` (err, the variable defined with the *verb tl*) could be called in the ⟨end-code⟩ of a verbatim environment (see 2.2), which would result in the verbatim content just grabbed (into `\g_codeverb_verb_body_tl`) being displayed using the *lstenv* environment (set up with the *listings* package). This will be, effectively, equivalent to

```
\begin{lstenv}
  ... content of \g_codeverb_verb_body_tl 'retokenized' ...
\end{lstenv}
```

Important: The verbatim code will be re-scanned (retokenized) in the process with the current category code régime.

`\cvRetokenNewCommand`	`\cvRetokenNewCommand` {⟨cmd⟩}
`\cvRetokenNewCommandV`	`\cvRetokenNewCommandV` {⟨cmd⟩}
`\codeverb_retoken_newcs:N`	`\codeverb_retoken_newcs:N` {⟨cmd⟩}
`\codeverb_retoken_newcsV:N`	`\codeverb_retoken_newcsV:N` {⟨cmd⟩}

`\cvRetokenNewCommand` and `\codeverb_retoken_newcs:N` will make ⟨cmd⟩ an alias to `\tl_retokenize:n` (from *expl3*, see [5]), while `\cvRetokenNewCommandV` and `\codeverb_retoken_newcsV:N` will make ⟨cmd⟩ an alias to `\tl_retokenize:V`. These commands are just for completeness (with the previous set of commands, which retokenize some verbatim content inside a given environment). Moreover, their main aim is to allow the code, grabbed as verbatim content, to be (re)used as LaTeX commands/text, the same overall idea from the commands `\getstored` (package *scontents*, see [2]) or `\tsexec` (package *codedescribe*, see [1]).

2.5 Auxiliary Display Commands

`\cvHideSpc`	`\cvHideSpc` {⟨text⟩}
`\codeverb_hide_spc:n`	`\codeverb_hide_spc:n` {⟨text⟩}

These will just replace every occurrence of a space (character code 32) by `\space`. They are the default commands used by `\cvSVerb` and `\codeverb_sverb:Nw` and are needed because (by default) spaces also have their category code changed to other (see 2.1), resulting that spaces are visible as in "example␣of␣visible␣spaces".

2.6 Short Verbatim Commands

The following commands aims at defining "short cuts" (active characters, as in other verbatim packages) allowing to grab short verbatim content and use it "right away", for instance, inside a paragraph. Unlike other verbatim packages, the following commands allow to customize the used characters category code, per command, as well to customize the action taken with the just grabbed verbatim content.

\cvSVerbSet	\cvSVerbSet [⟨keys⟩] {⟨code⟩}
\cvSVerb	\cvSVerb {⟨char⟩} ... ⟨verbatim content⟩...⟨char⟩
\codeverb_sverb_setcs:nn	\codeverb_sverb_setcs:nn {⟨keys⟩} {⟨code⟩}
\codeverb_sverb:Nw	\codeverb_sverb:Nw {⟨char⟩} ... ⟨verbatim content⟩...⟨char⟩

Both, \cvSVerb and \codeverb_sverb:Nw, are the immediate commands. They will use the character, ⟨char⟩, following them, as a marker and grab everything between it and its next appearance as verbatim content. ⟨code⟩ (set with \csSVerbSet or \codeverb_sverb_setcs:nn) will receive the verbatim content as #1. The default ⟨code⟩ is \codeverb_hide_spc:n {#1} "hidden" spaces.

> ***Note:*** The category code of ⟨char⟩ itself will be made other, regardless of the actual underlying category code in use whilst grabbing the content, the category code change being local.

With \csSVerbSet and \codeverb_sverb_setcs:nn one can customize the commands \cvSVerb and \codeverb_sverb:Nw. ⟨keys⟩ can be any of the common keys (see 2.1) and allow to change the effective category code table in use, whilst ⟨code⟩ will receive the grabbed content as #1. For instance, with \cvSVerbSet{#1} the content grabbed with \csSVerb will be presented "as is" and visible spaces. Likewise, \cvSVerbSet{ { \ttfamily #1 } } will result that the content grabbed with \csSVerb will be presented "as is" using the \ttfamily (note the extra set of braces to localize the effect of the \ttfamily command).

> ***Note:*** One can use \hlDisplay or \codehl_display:nn (see 3.3) when defining ⟨code⟩. Pay attention that those display commands expect the content to be terminated with a ^^J. Use, then, the *short verb* key (which will add a ^^J to the grabbed content), for instance: \cvSVerbSet { \hlDisplay[inline,short verb]{#1} }.

\cvSVerbNew	\cvSVerbNew [⟨keys⟩] {⟨start-patt⟩} {⟨end-patt⟩} {⟨code⟩}
\cvSVerbOn	\cvSVerbOn
\cvSVerbOff	\cvSVerbOff
\codeverb_sverb_new:nnnn	\codeverb_sverb_new:nnnn {⟨keys⟩} {⟨start-patt⟩} {⟨end-patt⟩} {⟨code⟩}
\codeverb_sverb_on:	\codeverb_sverb_on:
\codeverb_sverb_off:	\codeverb_sverb_off:

\cvSVerbNew and \codeverb_sverb_new:nnnn allow the setup and definition of custom short commands associated with active chars (characters with category code active, 13). Once defined, and activated in text, everything between ⟨start-patt⟩ and ⟨end-patt⟩ will be grabbed as verbatim content and processed by ⟨code⟩ (which will receive the verbatim content as #1. These can be called more than once, defining different ⟨star-patt⟩, ⟨end-patt⟩, and associated ⟨code⟩. \cvSVerbOn and \codeverb_sverb_on: turn those definitions on, while \cvSVerbOff and \codeverb_sverb_off: deactivate them. For example:

```
\ExplSyntaxOn
\codeverb_sverb_new:nnnn {} {||} {||}
  { \tl_trim_left_spaces:n { #1 } }
\ExplSyntaxOff

\cvSVerbOn
Example of use: || \cmd{#1}||
\cvSVerbOff
```

Example of use: \cmd{#1}

> ***Note:*** One can use \hlDisplay or \codehl_display:nn (see 3.3) when defining ⟨code⟩. Pay attention that those display commands expect the content to be terminated with a ^^J. Use, then, the *short verb* key (which will add a ^^J to the grabbed content), for instance: \cvSVerbNew {||}{||}{ \hlDisplay[inline,short verb]{#1} }.

2.6.1 Caveats

A few caveats to be considered:

- ⟨start-patt⟩ is a single (unique) character. This is the simplest case, the ⟨start-patt⟩ character will be made active and the content between it and ⟨end-patt⟩ will be collected as verbatim content.
- ⟨start-patt⟩ is a pattern compose of more than one character. In this case the 1st character of ⟨start-patt⟩ will be made active and look ahead to see if the following tokens match the

rest of the ⟨start-patt⟩, if so, then, the content between ⟨start-patt⟩ and ⟨end-patt⟩ will be collected as verbatim content.

- There are more than one ⟨start-patt⟩ with the same 1st character: This is very similar to the previous case, though, the active code (associated with the 1st character) will test for every possible ⟨start-patt⟩, starting by the last defined one.

  Note: For instance, one may want 3 definitions for ⟨start-patt⟩: `|`, `||` and `|||`, in which case the longest one should be defined last.

  Attention When looking ahead, `\peek_regex:NTF` is used. One side effect is that the first character following the ⟨start-patt⟩ will be grabbed with the current category code table (and not the verbatim one). In particular if the 1st character following ⟨start-patt⟩ is a special one, like `\` or `#`, this will, probably, lead to low level errors. The work around is to use a single space after the ⟨start-patt⟩, since this 1st space will be grabbed with the current category code it can be easily removed with a command like `\tl_trim_left_spaces:n` (as in the previous example) since it will (default settings) be the single space with category code 10 (space).

- The characters made active by the `\cvSVerbNew` and `\codeverb_sverb_new:nnnn`, while effectively actives (`\cvSVerbOn`), will be made other when using a verbatim environment (see 2.2) or using any of the short verbatim commands.

`\cvSVerbUndef`	`\cvSVerbUndef` {⟨start-patt⟩}
`\cvSVerbUndefAll`	`\cvSVerbUndefAll`
`\codeverb_sverb_undef:n`	`\codeverb_sverb_undef:n` {⟨start-patt⟩}
`\codeverb_sverb_undef_all:`	`\codeverb_sverb_undef_all:`

`\cvSVerbUndef` and `\codeverb_sverb_undef:n` will undefine the short cut associated with ⟨start-patt⟩, while `\cvSVerbUndefAll` and `\codeverb_sverb_undef_all:` will undefine all current defined short cuts.

3 codehighlight Package

After capturing some code in verbatim mode (see 2) this provides commands to typeset them, highlighting some structures/keywords in the process.

> ***Note:*** This is not a pretty print or formatter package. That for, a full parser would be necessary, which is way beyond the scope of this package.

The specification of how and what will be highlighted is split in:

- Language syntax (⟨lang⟩ from 3.12), which defines how the code scanner can recognize elements as identifiers, comments, strings, etc. and associate those recognized elements with a formatting class.
- Keywords sets (⟨kewy-set⟩ from 3.11), which just defines (list) sets of keywords, and associate them with a formatting class.
- Highlight sets (⟨hl-set⟩ from 3.9), which maps a formatting class with the formatting code itself.
- Base format sets (⟨format-set⟩ from 3.8) which sets the general formatting of the code to be displayed, like frame, background, etc.

All those aspects are decouple from each other, for instance one can use different sets of keywords with a given language, or have alternative highlight sets and use one or the other as desired.

As mentioned, the following display commands expect the code was captured by *codeverbatim* and stored in an *expl3* token list variable. The user is free to capture it in other ways, or process the code of said variable, in which case a few remarks are important:

- The end of line, line break mark is `^^J` (following the *expl3* [5] convention, see the section "Reassigning category codes", command `\tl_retokenize:n`, from *The LaTeX3 Interfaces* [5])
- No character with the following category codes should appear (of course, the characters themselves can appear, just not with the following category codes):

catcode	name	usually	catcode	name	usually
0	escape	`\`	6	parameter	`#`
1	begin group	`}`	7	math superscript	`^`
2	end group	`{`	8	math subscript	`_`
3	math toggle	`$`	14	comment	`%`
4	alignment	`&`			

3.1 UTF8 and other active characters

Characters with character code greater then 127 are assumed to be active `utf8` characters and "aren't touched": the scanning routines will keep then as is, together, letting LaTeX 2$_\varepsilon$ process them in the usual way.

On the other hand, characters with a character code below 128 are assumed to be non-active. In general, it's advisable to handle them before calling any of the display commands, either by turning them off before grabbing the code, or re-configuring *codeverbatim* (to turn them off) or processing the grabbed code before displaying it.

The main problem that might arise from an active character (category code 13) with a character code below 128 is that the highlighting routines, besides the formatting code itself, will insert lots of breaking commands (e.g. `\penalty0\relax` commands) that, most likely, won't work well with whatever code was associated with the active character, most likely triggering low level kernel errors.

Of course, if the code associated with a given active character doesn't depends on *what follows*, it is safe to preserve them as active characters.

3.2 Setting defaults

`\hlDispSettings` `\hlDispSettings {⟨keys⟩}`
`\codehl_display_settings:n` `\codehl_display_settings:n {⟨keys⟩}`

These will set (or change) the many display defaults. Note that, this is when the scan functions and variables associated with a language, keywords and highlight sets are created (see 3.6).

⟨keys⟩ can be any combination of:

`language` This will select the given language as the default one (See 3.12). If set to an empty value (*`language = {}`*) no language specific patterns will be recognized.

`hl set` This will select the given highlight set as the default one. See 3.9. If set to an empty value (*`hl set = {}`*) no syntax highlight will ensue.

`keywd set` This will select the given keyword set as the default one. See 3.11. If set to an empty value (*`keywd set = {}`*) no keyword will be recognized.

`base format` This will select the given base format set as the default one. See 3.8. If set to an empty value (*`base format = {}`*) then *`std-fmt`* (it only sets the *`text width`* and *`wrap indent`*) will be selected as the base formatting.

> ***Note:*** All keys can be set to an empty value, but, if a non empty value is given, and the respective *`language`*, *`hl set`*, *`keywords set`* or *`base format`* doesn't exist, an error will be raised.
> ***Note:*** Initially, *`language`* and *`keywd set`* are set to empty (no definition), whilst, *`base format`* is set to *`str-fmt`* and *`hl set`* is set to *`str-hlset`*.

3.3 Display commands

`\hlDisplay` `\hlDisplay [⟨keys⟩] {⟨code⟩}`
`\hlDisplayV` `\hlDisplayV [⟨keys⟩] {⟨tl-var⟩}`
`\codehl_display:nn` `\codehl_display:nn {⟨keys⟩} {⟨code⟩}`
`\codehl_display:no` `\codehl_display:no {⟨keys⟩} {⟨tl-var⟩}`

These will display (typeset) the contents of ⟨code⟩ or ⟨tl-var⟩ (an *`expl3`* token list variable) accordingly with the active language, highlight set, etc.

> ***Note:*** The highlight routines expect that ⟨code⟩ is terminated by `^^J` . A quark loop will ensue otherwise.

⟨keys⟩ can be any combination of the keys from 3.8 (except the *`based on`* key) and the following:

`hl set` This will locally extend the active highlight set. The expected syntax is *`hl set = {⟨hl-keys⟩}`*, whereas ⟨hl-keys⟩ can be any combination of keys from 3.9 (except the *`based on`* key).

> ***Note:*** This can only be used if a valid *`hl set`* was selected with `\hlDispSettings` or `\codehl_display_settings:n`.

`keywd set` This will locally extend the keywords set. The expected syntax is *`keywd set = {⟨keywd-keys⟩}`*, whereas ⟨keywd-keys⟩ can be any combination of keys from 3.11 (except the *`based on`* key).

> ***Note:*** This can only be used if a valid *`keywd set`* was selected with `\hlDispSettings` or `\codehl_display_settings:n`.

3.4 Language and syntax highlight definitions

There are a few main cases to be taken care of:

- comments/strings → A language ⟨lang⟩ (see 3.12) defines the start/end patterns and associated ⟨class⟩, whilst a highlight set ⟨hl-set⟩ (see 3.9) associates the related ⟨class⟩ with a format to be used.

  For example, when setting a language one can have: *`comment = {3}{/*}{*/}`* which will define a C++ style comment and associate *`comment`* ⟨class⟩ *`3`* with it. The format to be applied will come from a highlight set (see 3.9), like: *`comment = {3}{\color {blue}}`*.

  > ***Note:*** Comments can be discarded, or made *active*, meaning the comment text will be handled as normal LaTeX 2_ε code.

- keywords → A keyword set, ⟨keywd-set⟩ (see 3.11), associates keywords with a ⟨class⟩, whilst a highlight set *`hl-set`* (see 3.9) associates the related ⟨class⟩ with a format to be used.

  For example, when setting a keyword set, one can have: *`keywd = {2}{some,words}`* and *`keywd = {3}{special}`*. The format to be applied will come from a highlight set (see 3.9), like: *`keywd`*

`= {2}{\bfseries }` and `keywd = {3}{\color {red}}`. That way the words *some* and *words* will appear in bold face, whilst *special* in red.

- identifiers → `language` (see 3.12) defines the delimiter ⟨char⟩ and associated ⟨id-name⟩, a ⟨keywd-set⟩ (see 3.11) associates keywords with a ⟨id-name⟩⟨class⟩, and ⟨hl-set⟩ (see 3.9) associates the related ⟨id-name⟩⟨class⟩ with a format to be used.

  That's a special case of keywords, for example, when setting a language one can have: `identifier = {texcs}{\\}` which will define a LaTeX 2ε style identifier, associated with the ⟨id-name⟩ `texcs`.

  Then, when setting a keyword set, one can have: `texcs = {2}{cmdA,cmdB}` and `texcs = {3}{cmdC}`, and, finally, the format to be applied will come from a highlight set (see 3.9), like: `texcs = {2}{\bfseries }` and `texcs = {3}{\color {blue}}`. That way the commands `\cmdA` and `\cmdB` will appear in bold face, whilst `\cmdA` in blue.

  > ***Note:*** Note that, even though, all these aspects are related, they all (language sets, highlight sets, keyword sets and base format set) are decouple at definition time. Only when effectively displaying a given code that they will be "bring together", this allows (for instance) to have separated sets of keywords for a given language, and using one or another as needed.

3.5 Based on syntax and class remapping

In the following sections, a common key is `based on`, whose general syntax is:

`based on` = `{` ⟨base-ref⟩ `,` ⟨base-ref⟩ `...}`

And, ⟨base-ref⟩ can be any combination of:

⟨base⟩	The ⟨base⟩ will be inherited, as is. No class re-mapping.
`R[int]`⟨base⟩	The ⟨base⟩ classes will be re-mapped, adding ⟨int⟩ to all ⟨base⟩ classes.
`S[int]`⟨base⟩	The ⟨base⟩ classes will be re-mapped, setting all ⟨base⟩ classes to ⟨int⟩.

> ***Note:*** A ⟨base⟩ can be based on other ⟨bases⟩. Keep in mind, though, no attempt is made to detect a circular definition, which will end exhausting LaTeX 2ε memory.

As briefly presented in the introduction above, a key concept of all sets (syntax, keywords, highlight) is the idea of a ⟨class⟩, an integer, which allows to have many formatting ⟨classes⟩ associated with each element.

Regarding the `based on` key, when defining a set based on another one, one has the option to just copy the definitions as is, or *remap* the base classes, either setting them to a constant, `S[int]`⟨base⟩, or add a constant `R[int]`⟨base⟩ to them.

> ***Note:*** When processing the many sets (base format, see 3.5, highlight, see 3.9, keywords, see 3.11 or language, see 3.12) the `based on` keys are processed first in the definition order.
> Moreover, the `based on` definitions can be nested. For instance, one can have a language `langA` based on languages `langB` and `langC`, and `langB` based on `langD`, and so on.

3.6 Internal structures and scan functions

Prior effectively displaying/highlighting some code, a set of scan functions (derived from the language definition, see 3.12) and auxiliary variables (derived from keywords sets, see 3.11, highlight sets, see 3.9 and base format, see 3.8) need to be set. Note though:

- Those scan functions and variables will always be locally defined. This allows to, for instance, have a local set of keywords extending a base set, but not changing it.

Depending on the specifics, setting up the needed scan functions (for a language) and auxiliary variables can be a time consuming operation. That's why setting those functions/variables is delayed as much as possible, eventually to the first call of `\hlDispSettings` (see 3.2). Usually this is ok, but, if `\hlDispSettings` is called from inside a local group (like inside an environment) this can lead to the undesirable side effect of rebuilding those functions/variables time and time again.

3.7 Some package settings

\hlPackSettings `\hlPackSettings {⟨settings⟩}`
\codehl_pkg_settings:n `\codehl_pkg_settings:n {⟨settings⟩}`

This is meant to change some global settings. For now, ⟨settings⟩ can only be:

`suppress notes` (default: true) If set to true the *info/note* key from `\codehl_lang_def:nn` (see 3.12) and `\codehl_keyset_def:nn` (see 3.11) will be ignored. Note, the package's default is *suppress notes = false*.

3.8 Base Format

A base format defines general aspects of code highlighting, as framing, background color (if any), font, etc. By default, there is an initial set, *std-fmt*, which only sets the *text width* to *\linewidth* and *wrap indent* to *2em* (the full setup can be seen with `\hlBasefmtShow{std-fmt}`).

\hlBasefmtDef `\hlBasefmtDef {⟨format-set⟩} {⟨keys⟩}`
\codehl_basefmt_def:nn `\codehl_basefmt_def:nn {⟨format-set⟩} {⟨keys⟩}`

⟨keys⟩ can be any combination of:

`based on` *{ ⟨fmtset-ref⟩ , ⟨fmtset-ref⟩ ...}*
The definitions of ⟨fmtset-ref⟩ set(s) are inherited. See 3.5 for details.

`info/note` {⟨note⟩}
⟨note⟩ will be logged into the log file. This is merely informational and can be suppressed with the command `\codehl_pkg_settings` (see 3.7).

Behaviour keys:

`gobble` (default: *0*) The number of characters to be gobble from each line prior any processing.

> ***Note:*** There is no provision for utf8 chars here. It will fail badly if an utf8 character is present among the characters to be gobbled. The main purpose of this is to de-indent source code.

`inline` (default: *false*) Normally the code is displayed in vertical mode, constraining its width. With *inline* set to true, the code to be highlighted will be assumed to be short and displayed in its natural length.

> ***Note:*** A `^^J` found in the middle of ⟨code⟩ will result in a line break, except the one at the end of ⟨code⟩.

`short verb` (default: *false*) This will add a `^^J` at the end of ⟨code⟩ before it's further processed. This is needed (to avoid a quark loop) in case of verbatim code grabbed with one of the short verbatim commands (see 2.6), which doesn't terminate the grabbed code with a `^^J`.

Format keys:

`base format` This will be applied in a local group in which all highlighted code will be displayed. This shouldn't absorb any token.

`show spaces` (default: *false*) If spaces are shown or not.

`text width` (default: *\linewidth*) This set's the text width of the displayed code.

`wrap indent` (default: *2em*) The indentation amount to be used in case of wrapping lines.

`background` Set's the background color to be used. Can be any color as defined by *xcolor*. By default no background is applied (equivalent to an empty argument).

Line numbers keys:

`numbers` (default: *none*) Possible values: *none*, *left* and *right*. Line numbers will be attached (or not) to each line.

`numbers format` Format to be used when displaying line numbers. The used command(s) can absorb a single parameter, namely the line number.

`numbers sep` (default: *2pt*) The separation between the displayed text and the line numbers.

Frame keys:

`rule width` (default: *0.25pt*) The width of the rules (if any) when framing the text.

`rule color` (default: *black*) The color of the rules (if any) when framing the text.

`left frame` (default: *false*) If a rule at the left should be present.

`right frame` (default: *false*) If a rule at the right should be present.

`top frame` (default: *false*) If a rule at the top should be present.

`bottom frame` (default: *false*) If a rule at the bottom should be present.

`frame sep` (default: *2pt*) The separation between the frames (if any) and the text.

`left sep` (default: *0pt*) Extra separation (added to *frame sep* between the left frame and the text.

`right sep` (default: *0pt*) Extra separation (added to *frame sep* between the right frame and the text.

`top sep` (default: *0pt*) Extra separation (added to *frame sep* between the top frame and the text.

`bottom sep` (default: *0pt*) Extra separation (added to *frame sep* between the bottom frame and the text.

`\hlBasefmtShow` `\hlBasefmtShow {⟨format-set⟩}`
`\codehl_basefmt_show:n` `\codehl_basefmt_show:n {⟨format-set⟩}`

This will write to the terminal (raising an error message) the effective ⟨format-set⟩, after applying defaults and recovering all *based on* definitions.

3.9 Highlight Sets

A Highlight set defines the formatting to be applied to the many detected elements (as defined by language 3.12). The formats commands can absorb a single argument, namely the text/element detected and being formatted. By default, there is an initial set, *std-hlset*, which only sets *comment* (classes 1 to 6) to `\itshape` , *string* (classes 1 to 6) to `\slshape` and *keywd* and *texcs* (classes 1 to 6) to `\slshape` (the full setup can be seen with `\hlHLsetShow{std-hlset}`). *std-hlset* is just for convenience, allowing for a quick and simple setup (like in 4.2).

`\hlHLsetDef` `\hlHLsetDef {⟨hl-set⟩}{⟨keys⟩}`
`\codehl_hlset_def:nn` `\codehl_hlset_def:nn {⟨hl-set⟩}{⟨keys⟩}`

All keys (except for *based on*) expect two arguments: {⟨class⟩}{⟨fmt-cmds⟩}.

⟨fmt-cmds⟩ can absorb a single argument. The ⟨class⟩ (an integer) allows to have distinct formats for the same structure (be it comments, strings, keywords, etc...).

⟨keys⟩ can be any combination of:

`based on` *{ ⟨hlset-ref⟩ , ⟨hlset-ref⟩ ...}*
The definitions of a base ⟨hlset-ref⟩ set(s) are inherited. See 3.5 for details.

`info/note` {⟨note⟩}
⟨note⟩ will be logged into the log file. This is merely informational and can be suppressed with the command `\codehl_pkg_settings` (see 3.7).

`comment` {⟨class⟩}{⟨fmt-cmds⟩} To be applied to *comments*.

`string` {⟨class⟩}{⟨fmt-cmds⟩} To be applied to *strings*.

`keywd` {⟨class⟩}{⟨fmt-cmds⟩} To be applied to *keywords*.

`emph` {⟨class⟩}{⟨fmt-cmds⟩} To be applied to *keywords*, this is just an alternative way to format keywords (similar to *listings*[3] extra emphasis).

`active` {⟨class⟩}{⟨fmt-cmds⟩} To be applied to *active* comments.

⟨id-name⟩ {⟨class⟩}{⟨fmt-cmds⟩} To be applied to identifiers named ⟨id-name⟩. See 3.12 for details

⟨tag-name⟩ {⟨class⟩}{⟨fmt-cmds⟩} To be applied to keywords inside a *tagged area*. See 3.12 for details

Note: Since this is decoupled (at definition time) from language and keyword sets, any other *key* being used will be treated as a possible identifier (⟨id-name⟩ or ⟨tag-name⟩ see 3.12), and no error will be raised.

Note: If you want to delete the format associated with any given *key* ⟨class⟩ pair, just redefine them as nothing, e.g., *keywd = {2}{}*.

\hlHLsetShow	\hlHLsetShow {⟨hl-set⟩}
\codehl_hlset_show:n	\codehl_hlset_show:n {⟨hl-set⟩}

This will write to the terminal (raising an error message) the effective ⟨hl-set⟩ (after recovering all *based on* definitions).

3.10 Auxiliary Format Commands

\hlFmt	\hlFmt {⟨id⟩}{⟨class⟩}
\hlSetFmt	\hlSetFmt {⟨hl-set⟩}{⟨id⟩}{⟨class⟩}
\codehl_hlset_fmt:nn	\codehl_hlset_fmt:nn {⟨id⟩}{⟨class⟩}
\codehl_hlset_fmt:nnn	\codehl_hlset_fmt:nnn {⟨hl-set⟩}{⟨id⟩}{⟨class⟩}

These are meant to be used in the definition of ⟨fmt-cmds⟩, see 3.9. The versions with two parameters (⟨id⟩ and ⟨class⟩) use the current active set, whilst the ones with three parameters allow to reference the entries from a different highlight set. ⟨id⟩ can be any highlight set key, except *based on*: *comment*, *string*, *keywd*, *emph*, *active* or any *id-name* or *tag-name*. See 3.9.

Note: Since this is decoupled (at definition time) from any language, highlight set and/or keyword set, no error will be raised if referencing a non-existent ⟨hl-set⟩, ⟨id⟩ or ⟨class⟩.

Note: These commands can be used anywhere, given that the corresponding highlight set was already processed (see 3.2 and 3.6), though an *active set* is only well defined in the context of display commands, see 3.3.

3.11 Keywords Sets

Sets of *detectable* keywords. These defines the list of keywords and associated formatting ⟨class⟩.

\hlKeysetDef	\hlKeysetDef {⟨keywd-set⟩}{⟨keys⟩}
\codehl_keyset_def:nn	\codehl_keyset_def:nn {⟨keywd-set⟩}{⟨keys⟩}

All keys (except for *based on*, see 3.5) expect two arguments: {⟨class⟩}{⟨keywd-list⟩}. Besides that, one can add an asterisk to each keyy (for instance *keywd** or *emph**) in which case a list of keywords will be converted to lower case (using \str_lowercase:n).

The ⟨class⟩ (an integer) allows to have distinct sets of keywords, identifiers.

⟨keys⟩ can be any combination of:

based on The definitions of a base ⟨hl-set⟩ set are inherited. See 3.5 for details.

info/note {⟨note⟩}
⟨note⟩ will be logged into the log file. This is merely informational and can be suppressed with the command \codehl_pkg_settings (see 3.7).

keywd {⟨class⟩}{⟨keywords list⟩} List of keywords.

emph {⟨class⟩}{⟨keywords list⟩} List of keywords to be formatted as *emph*.

⟨id-name⟩ {⟨class⟩}{⟨keywords list⟩} List of ⟨id-name⟩ (*identifiers*) keywords to be formatted as *id-name* in 3.9. See *identifiers* from 3.12 for details

⟨tag-name⟩ {⟨class⟩}{⟨keywords list⟩} List of keywords, ⟨tag-name⟩, to be formatted as ⟨*tag-name*⟩ (see 3.9) inside a *tagged area*. See 3.12 for details

Note: Since this is decoupled (at definition time) from language and highlight format, any other *key* being used will be treated as a possible identifier (see 3.12), and no error will be raised.

`suppress` ⟨id-name⟩ {⟨class⟩}{⟨keywords list⟩} List of ⟨id-name⟩ keywords whose format (if already defined) will be suppressed. ⟨id-name⟩ can also be *keywd* or *emph*.

> ***Note:*** Since this is decoupled (at definition time) from language and highlight format, no error will be raised if a keyword wasn't already defined.
>
> ***Note:*** Again, each keyword can be postfixed with an asterisk, in which case the list of keywords will be converted to lower case (using `\str_lowercase:n`. This is meant for case insensitive languages.

`\hlKeysetShow`	`\hlKeysetShow` {⟨keywd-set⟩}
`\codehl_keyset_show:n`	`\codehl_keyset_show:n` {⟨keywd-set⟩}

This will write to the terminal (raising an error message) the effective ⟨keywd-set⟩ (after recovering all *based on* definitions).

3.12 Language Syntax

The elements of a language (for instance, comments, strings) are defined in terms of their delimiters. There are 4 main cases:

- keywords: any combination of letters. That's *implicit* for all languages.
- Identifiers: keywords preceded by a marker (⟨char⟩ below), like TEX commands.
- Fully delimited elements (those with ⟨start-pattern⟩ and ⟨end-pattern⟩ below), like C++ comments starting with `/*` and ending with `*/`, or strings.
- Line delimited elements (those with ⟨start-pattern⟩ below), like C single line comments starting with `//`.

> ***Note:*** When defining the patterns, don't forget to escape TEX special characters, specially the ones with category code besides *other* (12) and *letter* (11): 0: escape itself (usually `\`), 1: begin-group (usually `{`), 2: end-group (usually `}`), 3: math shift (usually `$`), 4: alignment tab (usually `&`), 6: parameter (usually `#`), 7: superscript (usually `^`) , 8: subscript (usually `_`) , 14: comment (usually `%`).
> In fact, all characters can be escaped, for instance `{\a\d\:\$}` is, effectively, `{ad:$}`
>
> ***Note:*** When processing the ⟨patterns⟩, no expansion will occur, and all characters in the given pattern will receive the category code other. For instance, given a pattern `{\cmdX}` the effective pattern will be `{cmdX}`.

All structures can be nested inside others, but only a few can *hold* them, namely: a *string* can be nested into *string* or *delimited string*, whilst a *comment* can be nested into *comment*, *comment line* or *comment line at*. The option *nestable* controls which ones can be detected (if nested).

In case of the *discard* variants, obviously a ⟨class⟩ isn't really needed (no formatting), but it's required so that its syntax is the same as the *comment* counter parts, which allows one to just add or remove a *discard* word, as needed.

When analysing the text to be highlighted, the listed patterns are grouped by their first letter, then the rest of the pattern is looked for in the *reverse* definition order, that means, for example, if one has two comment definitions, like *comment = {3}{/**}{**/}* followed by *comment = {1}{/*}{*/}* the `/**` will never be detected, always put the longest sequence last.

> ***Note:*** If two structures are defined with the same ⟨start-pattern⟩ the one defined last overwrites any previous definition for that ⟨start-pattern⟩.

`\hlLangDef`	`\hlLangDef` {⟨lang⟩}{⟨keys⟩}
`\codehl_lang_def:nn`	`\codehl_lang_def:nn` {⟨lang⟩}{⟨keys⟩}

In the following, ⟨options⟩ can any combination of the following:

`nestable` The construct will be detected if nested inside another likewise structure: comments inside comments, or strings/identifiers inside strings.

> ***Note:*** Any *comment* construct declared as *nestable* will be detected if inside *comment*. *comment line* or *comment line at*.
> A *string* or *identifier* construct declared as *nestable* will be detected if inside *string* or *delimited string*.

`tagged` The construct will only be detected inside a tagged area.

`non tagged` The construct will only be detected outside a tagged area.

> ***Note:*** If none is given, the construct will be detected by default (including inside a tagged area), but not inside other structures, like comments or strings.

⟨keys⟩ can be any combination of:

`based on` The definitions of a base ⟨lang⟩ are inherited. See 3.5 for details.

`info/note` {⟨note⟩}
⟨note⟩ will be logged into the log file. This is merely informational and can be suppressed with the command `\codehl_pkg_settings` (see 3.7).

Keywords/identifiers behaviour keys:

`keywd in string` (default: *false*) If keywords (and/or identifiers) are to be detected (and formatted) inside strings.

`case sensitive` (default: *true*) If keywords (and/or identifiers) are case sensitive or not.

> ***Note:*** If set to false (case insensitive) a 'keyword candidate' will be converted to lower case prior comparing it with any list of defined keywords, identifiers or tag names (see 3.11). This implies that, it will only work IF the keywords lists are made of lower case keywords.

`alphanumeric` (default: *false*) If set to false (default) keywords and/or identifiers may only contain letters. If set to true they may start with *oletter* chars (besides letters) and may also contain *oletter* or *digits* chars.

> ***Note:*** This might interfere, in some cases, with other patterns, suppressing them if they start with one of the characters listed as *digits* or *oletter*. The best approach would be to just capture those characters as letter (see 2.1 and 2.3).

`digits` {⟨dig-set⟩}
Sets the list of what a *digit* is. A regular expression composed of the range 0-9 plus ⟨dig-set⟩ will be constructed as `[0-9⟨dig-set⟩]` (see the section *Character classes* in *The LaTeX3 Interfaces* [5]). This only applies if, and only if, *alphanumeric* is set to true.

`digits*` {⟨dig-set⟩}
Sets the list of what a *digit* is. A regular expression composed only of ⟨dig-set⟩ will be constructed as `[⟨dig-set⟩]` (see the section *Character classes* in *The LaTeX3 Interfaces* [5]). This only applies if, and only if, *alphanumeric* is set to true.

`oletter` {⟨o-set⟩}
Sets the list of what a *oletter* is. A regular expression composed only of ⟨o-set⟩ will be constructed as `[⟨o-set⟩]` (see the section *Character classes* in *The LaTeX3 Interfaces* [5]). This only applies if, and only if, *alphanumeric* is set to true.

> ***Note:*** Regarding *alphanumeric*, *digits* and *oletter*, it is slightly better (performance) to just to set those as letter (category code 11) and avoid the extra overhead, setting *alphanumeric* to false.

Pattern keys:

`identifier` {⟨id-name⟩}{⟨id-char⟩}{⟨options⟩}
The keyword following ⟨id-char⟩ (including the ⟨id-char⟩ itself) will be formatted as defined by *id-name*⟨class⟩ (from ⟨hl-set⟩).

`tagged area` {⟨tag-name⟩}{⟨start-pattern⟩}{⟨end-pattern⟩}{⟨options⟩}
In the region enclosed by ⟨start-pattern⟩ and ⟨end-pattern⟩ the usual keywords (the ones defined using the key *keywd* in a ⟨keywd-set⟩, see 3.11) won't be detected but only the ones declared as ⟨tag-name⟩. This aims at languages like *html* or *php*. If a *tagged area* is declared as *nestable* it can appear inside a string (for instance, like *php*).

`comment` {⟨class⟩}{⟨start-pattern⟩}{⟨end-pattern⟩}{⟨options⟩}
Everything between ⟨start-pattern⟩ and ⟨end-pattern⟩, including them, will be formatted as defined by *comment*⟨class⟩ (from ⟨hl-set⟩).

comment line {⟨class⟩}{⟨start-pattern⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and the end of line, will be formatted as defined by *comment*⟨class⟩ (from ⟨hl-set⟩).

comment line at {⟨class⟩}{⟨start-pattern⟩}{⟨col⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and the end of line, will be formatted as defined by *comment*⟨class⟩ (from ⟨hl-set⟩), but only if the first character appears at column ⟨col⟩.

discard comment {⟨class⟩}{⟨start-pattern⟩}{⟨end-pattern⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and ⟨end-pattern⟩, including them, will be discarded. The ⟨class⟩ is here just so this has the same construct as *comment*.

discard comment line {⟨class⟩}{⟨start-pattern⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and the end of line will be discarded. The ⟨class⟩ is here just so this has the same construct as *comment line*.

discard comment line at {⟨class⟩}{⟨start-pattern⟩}{⟨col⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and the end of line will be discarded, but only if the first character appears at column ⟨col⟩. The ⟨class⟩ is here just so this has the same construct as *comment line at*.

active comment {⟨class⟩}{⟨start-pattern⟩}{⟨end-pattern⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and ⟨end-pattern⟩, excluding them, will be: 1) executed as LaTeX 2_ε commands. 2) formatted as defined by *active*⟨class⟩ (from ⟨hl-set⟩).

active comment line {⟨class⟩}{⟨start-pattern⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and the end of line will: 1) executed as LaTeX 2_ε commands. 2) formatted as defined by *active*⟨class⟩ (from ⟨hl-set⟩).

active comment line at {⟨class⟩}{⟨start-pattern⟩}{⟨col⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and the end of line will: 1) executed as LaTeX 2_ε commands. 2) formatted as defined by *active*⟨class⟩ (from ⟨hl-set⟩). But only if the first character appears at column ⟨col⟩.

string {⟨class⟩}{⟨pattern⟩}{⟨escape-char⟩}{⟨options⟩}

Everything between ⟨pattern⟩ and ⟨pattern⟩, including them, will be formatted as defined by *string*⟨class⟩ (from ⟨hl-set⟩). The ⟨escape-char⟩ assures the following character won't be interpreted as a pattern delimiter of any kind.

delimited string {⟨class⟩}{⟨start-pattern⟩}{⟨end-pattern⟩}{⟨escape-char⟩}{⟨options⟩}

Everything between ⟨start-pattern⟩ and ⟨end-pattern⟩, including them, will be formatted as defined by *string*⟨class⟩ (from ⟨hl-set⟩). The ⟨escape-char⟩ assures the following character won't be interpreted as a pattern delimiter of any kind.

replace {⟨org-str⟩}{⟨replacement⟩}

Any occurrence of ⟨org-str⟩ will be replaced by ⟨replacement⟩. Note that ⟨org-str⟩ will be processed as any ⟨pattern⟩ in the previous keys, in particular, any space in it will be removed. Moreover, the scan routines will only look after those ⟨org-str⟩ after scanning for other patterns, the main implication: if a ⟨org-str⟩ starts with the same character used in any other pattern (comments, strings, etc.) this won't be recognized at all.

replace* {⟨org-str⟩}{⟨replacement⟩}

Any occurrence of ⟨org-str⟩ will be replaced by ⟨replacement⟩. Note that ⟨org-str⟩ will be processed as any ⟨pattern⟩ in the previous keys, in particular, any space in it will be removed. Unlike the previous key (without the asterisk) ⟨org-str⟩ will be treated as any other pattern, keep in mind that, in this case, a ⟨org-str⟩ might suppress the recognition of some other pattern (any previously defined one, starting with the same characters, but longer).

suppress {⟨pattern⟩}

The structure whose ⟨start-pattern⟩ equals ⟨pattern⟩ will be suppressed: no scan function will be generated for it.

> ***Note:*** It won't detect if the ⟨pattern⟩ corresponds, or not, to an already defined structure, given that the ⟨start-pattern⟩ might be inherited from a *based on* ⟨lang⟩.

Note: Sequence matters: If, later on, a new structure is defined with the same ⟨start-pattern⟩, it will be normally processed (overwriting the *suppress* key).

`\hlLangShow` `\hlLangShow {⟨lang⟩}`
`\codehl_lang_show:n` `\codehl_lang_show:n {⟨lang⟩}`

This will write to the terminal (raising an error message) the effective ⟨lang⟩ definitions (after recovering all *based on* definitions).

3.13 Aliases

`\hlLangAlias` `\hlLangAlias {⟨alias⟩} {⟨lang⟩}`
`\codehl_lang_alias:nn` `\codehl_lang_alias:nn {⟨alias⟩} {⟨lang⟩}`

These commands define a set of aliases for a language. Notice, thought, that the internal representation of those aliases will always be exactly one level deep. For instance you can define 'B' as an alias for 'A' then 'C' as an alias to 'B', in which case, internally, 'C' will be just another alias for 'A'. More important, if, later on, 'B' is redefined as an alias for 'X', 'C' will remain an alias for 'A'.

Note: An error will be raised if ⟨alias⟩ is an already defined language (not an alias).

Note: The alias definition will be logged into the log file. This can be suppressed with the command `\codehl_pkg_settings` (see 3.7).

`\hlKeysetAlias` `\hlKeysetAlias {⟨alias⟩} {⟨keyset⟩}`
`\codehl_keyset_alias:nn` `\codehl_keyset_alias:nn {⟨alias⟩} {⟨keyset⟩}`

Same, for a keyword set.

`\hlHLsetAlias` `\hlHLsetAlias {⟨alias⟩} {⟨hlset⟩}`
`\codehl_hlset_alias:nn` `\codehl_hlset_alias:nn {⟨alias⟩} {⟨hlset⟩}`

Same, for a highlight set.

`\hlBasefmtAlias` `\hlBasefmtAlias {⟨alias⟩} {⟨basefmt⟩}`
`\codehl_basefmt_alias:nn` `\codehl_basefmt_alias:nn {⟨alias⟩} {⟨basefmt⟩}`

Same, for a base format set.

3.14 Pre-defined Languages

There is a set of pre-defined languages known by *codehighlight*, but since it would be wasteful to load all definitions all the time, those language definitions are *stored* in auxiliary files.

Those auxiliary files will be loaded as needed, for instance when one tries to expand the definitions of a pre-defined language (`\codehl_lang_def:nn`, see 3.12), or defines a language based on pre-defined one (key *based on*, see 3.5), or sets the display defaults (`\codehl_display_settings:n`, see 3.2).

Note: File loading, can only occurs in the preamble. An error will be raised otherwise.

Attention: When defining a language, keyword set, highlight set, all definitions are local. This allows to, for instance, locally expand a keyword list, but it also has a side effect that, if an auxiliary file is loaded in a local group, the definitions will be lost.

The following commands, force the load of the needed auxiliary files, and should be used in the preamble. Note, though, that there isn't a one-to-one relationship between auxiliary packages and language/keyword sets, since some auxiliary packages might entice more than one language definition. Lastly, those commands keep track of which auxiliary files were already loaded, so it's safe to call them more than once for the same language/keyword set.

`\hlLangLoad` `\hlLangLoad {⟨lang-list⟩}`
`\codehl_lang_preload:n` `\codehl_lang_preload:n {⟨lang-list⟩}`

This will include the needed auxiliary packages. Whereas ⟨lang-list⟩ is a comma separated list of languages.

\hlKeysetLoad `\hlKeysetLoad {⟨keyset-list⟩}`
\codehl_keyset_preload:n `\codehl_keyset_preload:n {⟨keyset-list⟩}`

Similarly, this will include the needed auxiliary packages. Whereas ⟨keyset-list⟩ is a comma separated list of keyset sets.

The following table contain the current list of known languages and keyword sets. Languages like Fortran also have a pre-defined alias (in this case, just Fortran) normally pointing to the 'most recent' definition. Note: if a new variant is introduced, the alias shall point to it.

Language	Language sets	Keywords sets	Alias / Obs.
C	C.89 C.99 C.11 C.17 C.23	C.89 C.99 C.11 C.17 C.23	C → C.23
C++	C++.98 C++.11 C++.20 C++.23	C++.98 C++.11 C++.20 C++.23	C++ → C++.23
Csharp	Csharp.1 Csharp.6 Csharp.11 Csharp.13	Csharp.1 Csharp.6 Csharp.11 Csharp.13	Csharp → Csharp.13
java	java.SE25	java.SE25	java → java.SE25
python	python.2.7 python.3.0 python.3.6 python.3.12	python.2.7 python.3.0 python.3.6 python.3.12	python → python.3.12
SQL	SQL.92 SQL.23 PL/SQL	SQL.92 SQL.23 SQL.PostgreSQL SQL.MySQL PL/SQL	SQL → SQL.23
perl	perl	perl	
lua	lua.5.5	lua.5.5	lua → lua.5.5
html	html.5	html.5	html → html.5
php	php.8	php.8 php.mysqli php.pgsql php.xml-json php.date	php → php.8
TeX	TeX	TeX.ams TeX.babel TeX.bibtex TeX.hyperref TeX.kernel TeX.pgf TeX.pgfplots TeX.xpacks	from *codedescribe*
TeX	TeX.l3	TeX.l3ams TeX.l3bibtex TeX.l3exp TeX.l3kernel TeX.l3pgf	from *codedescribe*. cmds contain _
TeX	TeX.l3sign	TeX.l3amssign TeX.l3bibtexsign TeX.l3expsign TeX.l3kernelsign TeX.l3pgfsign	from *codedescribe*. cmds contain _:

4 Examples

In the following examples, for simplicity, mostly LaTeX 2$_\varepsilon$ commands are used.

4.1 A minimal construct

A minimal construct could entice loading a pre-defined language (C in the example), setting the display settings to only recognize the keywords (therefore, no comment or string will be recognized), and define an environment which would display the collected verbatim code right away:

```
% This in the preamble only
\hlLangLoad{C}

% once the file definition was loaded, the following can be anywhere
\hlDispSettings
  {
    keywd set = C
  }

\cvNewEnvironment[verb tl=\DemoV]{DemoEnv}{}
  {}
  { \hlDisplayV \DemoV }
```

```
  \begin{DemoEnv}
    for ( i=0 , i<10 , i++ ) {
      g += i;
    }
    // a comment
  \end{DemoEnv}
```

for (i=0 , i<10 , i++) {
  g += i;
}
// a comment

Changing the settings to include the language syntax:

```
\hlDispSettings
  {
    language = C ,
    keywd set = C
  }
```

```
  \begin{DemoEnv}
    for ( i=0 , i<10 , i++ ) {
      g += i;
    }
    // a comment
  \end{DemoEnv}
```

for (i=0 , i<10 , i++) {
  g += i;
}
// a comment

4.2 Customizing the language

One can change an existing language definition, for instance, adding a LaTeX escape (the *nestable* key is to allow it to be detected even inside another comment):

```
\hlLangDef{C}
  {
    active comment line = {1}{\$\$}{ nestable }
  }
```

```
  \begin{DemoEnv}
    for ( i=0 , i<10 , i++ ) {
      g += i;
    }
    // a comment $$ \textbf{this}
  \end{DemoEnv}
```

for (i=0 , i<10 , i++) {
  g += i;
}
// a comment ***this***

4.3 Further customization

One can use color to call attention to some parts of the code, in general this is achieved by customizing a ⟨hlset⟩ (see 3.9). Note that, in the following example, ⟨std-hlset⟩ is a pre-defined ⟨hlset⟩ (the default one, if none is declared).

```
\hlLangDef{C}
  {
    comment line = {2}{//+}{ }
  }
\hlHLsetDef{std-hlset}
  {
    comment = {2}{\itshape\color{red}}
  }
```

```
  \begin{DemoEnv}
    for ( i=0 , i<10 , i++ ) {
      g += i;
    }
    // a comment $$ \textbf{this}
    //+ specially important comment
  \end{DemoEnv}
```

```
for ( i=0 , i<10 , i++ ) {
  g += i;
}
// a comment this
//+ specially important comment
```

4.4 Using sequences

Note, the following example could be written in LaTeX 2_ε, but, it's more natural to write it in *expl3* (given the use of sequences). First a dedicated store command is created (see 2.4) then an environment using it (see 2.2).

```
\ExplSyntaxOn

  \codeverb_new_store_cmds:nn { } { DemoStore }
  \cs_generate_variant:Nn \DemoStore_gput_right:nn { V }

  \codeverb_new_environment:nnnnn
    {
      drop ~ first,
      inline ,       % The environment end code will receive the verbatim code as #1
      other =  { \^^K } ,
    }
    {DemoStore} {O{default}}
    { %% Here, #1 is the environment optional parameter
      \tl_set:Nn \DemoStore_tmp_tl {#1}
    }
    { %% Here, #1 is the collected verbatim code
      \DemoStore_gput_right:Vn
          \DemoStore_tmp_tl
          { #1 }
    }
\ExplSyntaxOff
```

Note that, the above code defines a new environment, *DemoStore*, which expects a single optional parameter: the 'sequence name' to be used. Once a few verbatim snips are collected, they can be merged (command `\DemoStoreMerge` below, created by the command `\codeverb_new_store_cmds:nn` above) and then displayed as a simple peace.

```
\begin{DemoStore}[sampleA]
      for ( i=0 , i<10 , i++ ) {
      g += i;
    }    // a comment $$ \textbf{this}
\end{DemoStore}

\begin{DemoStore}[sampleA]
      for ( j=0 , j<10 , j++ ) {
      h += i;
    }    // a comment $$ \textbf{this too}
\end{DemoStore}

\DemoStoreMerge {\mymerged} {sampleA} {1-2}
\hlDisplayV \mymerged
```

```
for ( i=0 , i<10 , i++ ) {
  g += i;
}    // a comment this
for ( j=0 , j<10 , j++ ) {
  h += i;
}    // a comment this too
```

5 To do, Future plans

These are some aspects that are either planned or that deserve some consideration:

- grabbing code from an external file (codeverbatim): defining a 'text marker' (begging/end ... label).
- optimize the background color code. It works as is, but, it's slow.
- "here code" typical syntax <<Label or <<'Label'. Possible approach: _scanT collects the next label and sets _end_regex
- handling of tabs (? fixed spacing or column mod num ?)
- 'expanded' identifiers: PL/SQL has some odd keywords starting with `SQL%`
- indexing might be implement in a similar way as it is in *codedescribe*.
- allow for further customization of line numbering: e.g. one every 5, or starting number, etc.
- simula/algol comments peculiar syntax.
- listings *procnamekeys* might be implemented. Idea: add another 'list' (like identifiers) whose 'result' would be setting a flag for the following keyword.
- writing code to an external file, albeit fairly simple, isn't planned for the time being.
- improve the examples section.

References

[1] Alceu Frigeri. *The codedescribe and codelisting Packages*. 2026, p. 21. URL: https://mirrors.ctan.org/macros/latex/contrib/codedescribe/doc/codedescribe.pdf (visited on 08/07/2026).

[2] Pablo González. *SCONTENTS - Stores LaTeX Contents*. 2024, p. 48. URL: http://mirrors.ctan.org/macros/latex/contrib/scontents/scontents.pdf (visited on 03/10/2025).

[3] Jobst Hoffmann. *The Listings Package*. 2024, p. 65. URL: http://mirrors.ctan.org/macros/latex/contrib/listings/listings.pdf (visited on 03/10/2025).

[4] The LaTeX Team. *LaTeX for authors. current version*. 2026, p. 27. URL: https://mirrors.ctan.org/macros/latex/base/usrguide.pdf (visited on 08/07/2026).

[5] The LaTeX Team. *The LaTeX3 Interfaces. The LaTeX Project*. 2026, p. 385. URL: https://mirrors.ctan.org/macros/latex/required/l3kernel/interface3.pdf (visited on 08/07/2026).

[6] Timothy Van Zandt. *The 'fancyvrb' package. Fancy Verbatims in LATEX*. 2026, p. 24. URL: http://mirrors.ctan.org/macros/latex/contrib/fancyvrb/doc/fancyvrb-doc.pdf (visited on 06/19/2026).